Performance Optimization of Matrix Multiplication on CPU and GPU Architectures

Implementation and Benchmarking on Single-Core CPU, Multi-Core CPU, GPU, and Tensor Cores

A report submitted in partial fulfillment of the requirements for

the

RESEARCH INTERNSHIP

at

INDIAN INSTITUTE OF TECHNOLOGY PALAKKAD

By

Akhilesh Mohan
Anzin Mohammed
Fidha Fathima
Maria Davis

Under the guidance of

Prof. Unnikrishnan C

INDIAN INSTITUTE OF TECHNOLOGY PALAKKAD

Department Of Computer Science And Engineering

Indian Institute of Technology Palakkad

Palakkad, Kerala, India

February 4, 2026

Acknowledgement

We would like to express our sincere gratitude to Prof. Unnikrishnan C. for his invaluable guidance, constant encouragement, and technical insights throughout the course of this research internship. His support helped us understand not only the theoretical aspects of high-performance computing but also the practical challenges involved in performance optimization on modern hardware architectures.

We extend our thanks to the Department of Computer Science and Engineering, Indian Institute of Technology Palakkad, for providing us with the opportunity and academic guidance to carry out this work. The research-oriented discussions and mentorship greatly contributed to the successful completion of this project.

We are also grateful to the faculty members and research scholars who provided suggestions and discussions that helped refine our approach. Special appreciation goes to our peers and colleagues for their cooperation, feedback, and support during experimentation and benchmarking.

Finally, we thank our families for their continuous motivation and support, which enabled us to dedicate our time and effort to this internship project.

Abstract

Throughout this internship, we learned that matrix multiplication performance depends not only on the number of computations but also on how data is accessed and how well the hardware is utilized. What initially seemed like a simple triple-loop algorithm became a deeper study of memory behavior and parallelism. In the CPU phases, we understood how cache locality, loop ordering, and blocking can greatly affect runtime even when the algorithm is unchanged. Moving to multicore programming showed us the benefits and challenges of dividing work across threads. When we shifted to GPUs, we saw how massive parallelism and shared memory can accelerate large computations, and finally, Tensor Cores helped us realize how specialized hardware and mixed precision can deliver extremely high performance for matrix operations.

Overall, this internship gave us practical insight into performance optimization and helped us connect theoretical concepts from computer architecture and parallel computing to real experimental results.

Table of Contents

Introduction

Matrix multiplication is one of the most common operations in computing and appears in many areas such as machine learning, scientific simulations, data processing, and computer graphics. Because it is used so widely and often on large datasets, its performance has a direct impact on how fast many real-world applications run.

At first glance, matrix multiplication looks simple to implement. However, during this project we realized that writing a correct program is very different from writing a fast one. The runtime increases rapidly with matrix size, and the way data is accessed in memory plays a major role in performance. Modern processors can perform arithmetic very quickly, but memory access is comparatively slow. This makes cache usage and data locality extremely important.

In this internship, we explored how the same matrix multiplication operation performs on different hardware platforms and how optimization techniques change the results. We started with a basic single-core CPU implementation and then applied improvements such as loop interchanging, transposition, and blocking to make better use of the cache. Through this, we learned how memory access patterns can significantly affect execution time.

Next, we moved to multi-core CPUs using OpenMP, where we parallelized the computation across multiple threads. This helped us understand practical issues like thread scaling and the limits of CPU parallelism.

After observing CPU limitations for large matrices, we implemented matrix multiplication on GPUs using CUDA. Here, we experienced how GPUs handle thousands of threads in parallel and how shared memory can be used to reduce slow global memory accesses. Finally, we implemented a Tensor Core version using the WMMA API, which showed us how specialized hardware can dramatically accelerate matrix operations.

Throughout the project, we benchmarked different matrix sizes and data types (float and double) and measured performance using execution time and GFLOPS. More importantly, this project helped us understand that performance optimization is not only about computation but also about memory behavior and parallelism

Overall, this work reflects our learning journey in understanding how the same mathematical operation behaves differently across single-core CPUs, multi-core CPUs, GPUs, and Tensor Core hardware.

Problem Statement

Matrix multiplication is a core operation used in many computational fields such as machine learning, scientific computing, and data processing. As matrix sizes grow, the computational cost increases rapidly, and naive implementations become inefficient for practical use. This creates a need for optimized implementations that can handle large matrices within reasonable time.

The objective of this project is to:

Implement matrix multiplication on a single-core CPU, multi-core CPU, GPU, and Tensor Cores within the GPU, and to benchmark the program for different input sizes and data types (int32, float, and double).

Phase 1

IMPLEMENTATION OF MATRIX MULTIPLICATION ON SINGLE CORE CPU

Introduction

In this phase, we implemented matrix multiplication on a single-core CPU to understand how memory access affects performance. Even though the algorithm is simple, its speed depends a lot on how data is read and reused from memory.

We compare them based on:

- How the algorithm works
- How well it uses memory (spatial and temporal locality)
- Their time and space complexity
- What performance optimizations can be applied
- How simply changing the order of loops can make huge differences in speed because of cache efficiency.

What Problem Does This Solve?

The main challenge in matrix multiplication is not only the number of arithmetic operations—but **how efficiently data is accessed in memory**.

Modern CPUs are fast, but memory access is slow. So the real performance problem is how we read and reuse data, not just how much we compute.

In this project, we try to fix this by:

- **Improving cache locality**
- **Reducing cache misses**
- **Optimizing loop structure**
- **Rearranging data for sequential memory access**

In this phase, we focused on improving cache usage by optimizing loop structure and data access patterns. Better locality reduces cache misses and leads to faster execution.

createMatrix() Algorithm

Goal:

Generate a matrix filled with random values in a deterministic and reproducible way.

How the Algorithm Works

- Allocates a 1D vector of size rows × cols.
- Uses **mt19937** (Mersenne Twister) for fast pseudorandom number generation.
- Generates values from **uniform distribution [-5, 5]**.
- Stores values in **row-major format**, meaning:
- A[i][j] = A[i * cols + j]

Pseudo Code

```
function createMatrix(rows, cols, seed):
  allocate vector M of size rows * cols
  initialize rng with seed
  for each index i in M:
    M[i] = random number in [-5, 5]
  return M
```

How Matrix Multiplication Works

Given:

- A = m × n
- B = n × p
- C = m × p

Each output element:

C[i][j] = Σ over k (A[i][k] * B[k][j])

The implementation optimizes *how we access A and B*, which determines:

- CPU cache usage
- Runtime performance

- Memory bandwidth efficiency

Code Implementation

Naive Method (i → j → k)

The naive algorithm is the simplest and most direct method for matrix multiplication.It uses three nested loops to compute each element of the output matrix. It also serves as the baseline to compare improved algorithms.

Working

- For every row i and column j, calculate the dot product with all k.
- A's row access is sequential (good).
- B's column access is **non-sequential and cache-poor**.

Pseudo Code

Input: Matrix A (size m × n), Matrix B (size n × p)

Output: Matrix C (size m × p) where C = A × B

Step 1: Initialize result matrix C with all zeros

Step 2: FOR each row i from 0 to m:

FOR each column j from 0 to p:

Initialize sum = 0

FOR each index k from 0 to n:

sum = sum + A[i][k] × B[k][j]

Set C[i][j] = sum

Step 3: Return matrix C

Memory Locality

Access Pattern	Locality
A row access	Good (sequential)
B column access	**Bad** (strided, cache misses)

Interchanged Method (i → k → j)

The loop-interchanged method is simply a smarter rearrangement of the naive matrix multiplication loops.In the naive approach, we repeatedly fetch the same values from matrix A many times, which wastes time and cache space. In this improved version, we change the order of the loops so that once we pick an element from A, we immediately use it everywhere it is needed before moving on.

Working

- Swap inner loops so that each A[i][k] is reused for all j.
- Now B is accessed row-wise → **cache-friendly**.

Pseudo Code

Input: Matrix A (size m × n), Matrix B (size n × p)

Output: Matrix C (size m × p) where C = A × B

Step 1: Initialize result matrix C with all zeros

Step 2: FOR each row i from 0 to m:

FOR each index k from 0 to n:

Load value a = A[i][k]

FOR each column j from 0 to p:

C[i][j] = C[i][j] + a × B[k][j]

Step 3: Return matrix C

Memory Locality

Access Pattern	Locality
A row	Good
B row (instead of column)	**Very good**
C row updated sequentially	Excellent

This is significantly faster than naive.

Transpose-Based Method

One of the biggest problems in the naive multiplication method is that matrix B is accessed column by column, which is slow because memory is arranged row by row.
Here we turn the columns of B into rows by transposing it. After transposing, the multiplication becomes much more memory-friendly.

Working

1. Transpose B into Bt so row access replaces column access.
2. Multiply using the form:
3. C[i][j] = dot(A[i,*], Bt[j,*])

Pseudo Code

Input: Matrix A (size m × n), Matrix B (size n × p)

Output: Matrix C (size m × p) where C = A × B

Step 1: Compute transpose BT of matrix B (size p × n)

Step 2: Initialize result matrix C with all zeros

Step 3: FOR each row i from 0 to m:

FOR each column j from 0 to p:

Initialize sum = 0

FOR each index k from 0 to n:

sum = sum + A[i][k] × BT[j][k]

Set C[i][j] = sum

Step 4: Return matrix C

Memory Locality

Access Pattern	Locality
A row	Excellent
Bt row	Excellent (formerly B column)
C row writes	Excellent

This method changes the way we access matrix B so that we read it in straight, continuous rows instead of jumping around in memory. This makes the CPU access data faster and improves performance.

Special Optimizations in the Code

1. Pointer Aliasing Minimization

const T* arow = &A[i*n];

constT* brow = &B[k*p];

T* crow = &C[i*p];

This reduces repeated index computations.

2. Row-Major Data Access

All loops are arranged to ensure **sequential memory scanning**.

3. Transpose to Fix Poor Locality

Instead of accessing B column-wise, we precompute Bt:

Bt[j*n + k] = B[k*p + j];

4. Avoiding Dynamic Allocations Inside Loops

All vectors (C, Bt) are preallocated.

5. Improved Arithmetic Intensity

Interchanged & transpose methods minimize repeated multiplications and maximize data reuse

Blocked Matrix Multiplication Algorithm with Autotuning

Introduction

The blocked matrix multiplication algorithm and associated autotuning mechanism are described in this report. Although matrix multiplication is a basic computing process, the straightforward method is quite slow for big matrices. By using computer memory more intelligently, the blocked algorithm makes it significantly faster.

What Problem Does This Solve?

The basic approach of multiplying two huge matrices requires the computer to retrieve data from memory millions of times. When compared to calculations, memory access is slow. This is solved by the blocked algorithm, which speeds up the entire process by reusing data that has previously been loaded into the fast memory, also known as the cache.

Blocked Algorithm

Basic Concept

Instead of multiplying entire rows and columns at once, the blocked algorithm divides the matrices into smaller square pieces called **blocks**.

Benefits of using blocks are:

- Small blocks fit into the computer's fast cache memory
- Once data is in cache, we can reuse it many times
- This reduces the number of slow memory accesses

How the Algorithm Works

The algorithm uses six nested loops organized in two groups:

Outer loops (blocking loops):

1. Loop through blocks of rows in matrix A (using variable `ii`)
2. Loop through blocks of columns in matrix A and rows in matrix B (using variable `kk`)
3. Loop through blocks of columns in matrix B (using variable `jj`)

Inner loops (computation loops):

4. Loop through individual rows within the current block (using variable `i`)
5. Loop through individual columns within the current block (using variable `k`)
6. Loop through individual elements to do the actual multiplication (using variable `j`)

Pseudocode

Algorithm: Blocked Matrix Multiplication

Input: Matrix A (size m × n), Matrix B (size n × p), Block size bs
Output: Matrix C (size m × p) where C = A × B

Step 1: Initialize result matrix C with all zeros

Step 2: FOR each block row (ii from 0 to m, jumping by bs):
FOR each block column/row (kk from 0 to n, jumping by bs):
FOR each block column (jj from 0 to p, jumping by bs):

```
Calculate block boundaries:
  i_max = minimum of (ii + bs, m)
  k_max = minimum of (kk + bs, n)
  j_max = minimum of (jj + bs, p)

FOR each row i from ii to i_max:
      FOR each column k
from kk to k_max: Load value a =
A[i][k]

                FOR each column j from
  jj to j_max: C[i][j] = C[i][j] + a ×
  B[k][j]
```

Step 3: Return matrix C

Code Implementation

The code has **two variants** of the blocked algorithm:

1. Blocked Naive Algorithm

This version applies blocking to the naive (i-j-k) loop order:

```
template<typename T>

vector<T> matmul_blocked_naive(const vector<T>& A, const vector<T>& B,
int m, int n, int p, int bs = 128) {
vector<T> C(static_cast<size_t>(m) * p, 0);

// Outer loops - work on blocks
for (int ii = 0; ii < m; ii += bs)
for (int kk = 0; kk < n; kk += bs)
for (int jj = 0; jj < p; jj += bs) {

    // Handle edges when block doesn't fit
perfectly int i_max = min(ii + bs, m);
    int k_max = min(kk + bs,
n); int j_max = min(jj + bs,
p);

    // Inner loops - naive order (i-
k-j) for (int i = ii; i < i_max;
++i) {
        T* crow = &C[(size_t)i * p +
  jj]; const T* arow = &A[(size_t)i
  * n];

        for (int k = kk; k <
```

```
                int j = 0;

                // Process 4 elements at once
        (unrolled loop) for (; j + 3 < J; j += 4) {
                    crow[j]   += a *
          brow[j]; crow[j + 1] += a *
          brow[j + 1]; crow[j + 2] += a *
          brow[j + 2]; crow[j + 3] += a *
          brow[j + 3];
                }

                // Process remaining
        elements for (; j < J; ++j)
                    crow[j] += a * brow[j];
            }
        }
    }
```

return C;
}

2. Blocked Interchanged Algorithm

In this method we combine blocking with loop interchange for better performance:

template<typename T>
vector<T> matmul_blocked_interchange(const vector<T>& A, const vector<T>& B, int m, int n, int p, int bs) {
vector<T> C(static_cast<size_t>(m) * p, 0);

// pointers for faster indexing
const T* Ap = A.data();
const T* Bp = B.data();
T* Cp = C.data();

// Outer loops - iterate over tiles
for (int ii = 0; ii < m; ii += bs) {
int i_max = min(ii + bs, m);
for (int kk = 0; kk < n; kk += bs) {
int k_max = min(kk + bs,
n);
for (int jj = 0; jj < p; jj += bs) {
int j_max = min(jj + bs, p);

```
        // Inner loops - interchanged order (i-
    k-j) for (int i = ii; i < i_max; ++i) {
            T* crow = Cp + (size_t)i * p
      + jj; const T* arow = Ap +
      (size_t)i * n;

            for (int k = kk; k < k_max; ++k) {
                T a = arow[k]; // Load A[i][k]
        once const T* brow = Bp + (size_t)k * p
        + jj; int J = j_max - jj;
                int j = 0;

                // Unroll by 4 (reduces loop
```

```
          crow[j]     += a * brow[j];
            crow[j + 1] += a * brow[j +
          1];
            crow[j + 2] += a * brow[j +
          2]; crow[j + 3] += a * brow[j +
          3];
        }
      }
    }
  }
}
```

}
return C;
}

Special Optimizations in the Code

1. Using Pointers

- Instead of accessing elements like `C[i][j]`, the code uses pointers (`T* crow`)
- This is faster because it avoids recalculating array positions repeatedly

2. Loading Data Once

- The value `A[i][k]` is loaded into variable `a` and reused multiple times
- This reduces memory reads significantly

3. Loop Unrolling

- The innermost loop processes 4 elements at once
- This reduces the overhead of loop control (checking conditions, incrementing counters)
- It also helps the processor work on multiple operations simultaneously

4. Boundary Checking

- The code handles cases where matrix dimensions aren't perfectly divisible by the block size
- Variables `i_max`, `k_max`, `j_max` ensure we don't go out of bounds

5. Raw Pointers (Blocked Interchanged Only)

- Uses `A.data()`, `B.data()`, `C.data()` to get raw pointers
- Provides even faster indexing compared to vector indexing
- Gives compiler more optimization opportunities

6. Two Optimization Levels

- **Blocked Naive**: Applies blocking to standard loop order
- **Blocked Interchanged**: Combines blocking + loop interchange for maximum performance

The Autotuning System

Why Autotuning is Needed?

The best block size depends on the specific computer hardware:

- Cache sizes (L1, L2, L3 caches)
- Cache line size
- Processor architecture

A block size that works well on one computer might be slow on another. Autotuning finds the best block size by testing different options.

Working of Autotuning

The autotuning system tests several block sizes and picks the fastest one.

Block sizes tested: 16, 32, 48, 64, 96, 128

Why these sizes?

- **16**: Minimum useful size, but might be too small
- **32, 48, 64**: Good middle ground, often optimal
- **96, 128**: Larger blocks, might be too big for cache

Autotuning Code

The code includes **separate autotuning** for both blocked algorithms:

Autotuning for Blocked Naive

int best_bs_naive = 128; // Start with default
{
cout << "Autotuning block size for matmul_blocked_naive...\n";
vector<int> cands = {16, 32, 48, 64, 96, 128};
double best_time = 1e308; // Very large number

// Try each block size
for (int bs : cands) {
auto t0 = HRClock::now();
auto Ctry = matmul_blocked_naive(A, B, m, n, p, bs);
auto t1 = HRClock::now();
double s = sec_between(t0, t1);

```
cout << " bs=" << bs << " -> " << s << " s\n";

// Remember the fastest one
if (s < best_time) {
    best_time = s;
  best_bs_naive = bs;
}

// Verify correctness
if (!almost_equal(C_naive, Ctry)) {
```

```
    cerr << "Autotune (naive): result mismatch at bs=" << bs << "\n";
  return 1;
}
```

}
cout << "Autotune (naive) picked bs = " << best_bs_naive << "\n";
}

Autotuning for Blocked Interchanged

int best_bs_interchange = 128; // Start with default
{
cout << "Autotuning block size for matmul_blocked_interchange...\n";
vector<int> cands = {16, 32, 48, 64, 96, 128};
double best_time = 1e308;

// Try each block size
for (int bs : cands) {
auto t0 = HRClock::now();
auto Ctry = matmul_blocked_interchange(A, B, m, n, p, bs);
auto t1 = HRClock::now();
double s = sec_between(t0, t1);

```
cout << " bs=" << bs << " -> " << s << " s\n";

// Remember the fastest one
if (s < best_time) {
    best_time = s;
  best_bs_interchange = bs;
}

// Verify correctness
if (!almost_equal(C_naive, Ctry)) {
  cerr << "Autotune (interchanged): result mismatch at bs=" << bs <<
"\n";
    return 1;
}
```

}
cout << "Autotune (interchanged) picked bs = " << best_bs_interchange << "\n";
}

Autotuning Algorithms

Algorithm 1: Autotuning for Blocked Naive

Input: Matrices A and B, List of candidate block sizes
Output: Optimal block size for blocked naive algorithm

Step 1: Create list of block sizes to test: [16, 32, 48, 64, 96, 128]
Step 2: Set best_time to infinity
Step 3: Set best_bs_naive to 128 (default)

Step 4: FOR each candidate block size:
Start timer
Run matmul_blocked_naive with this block size
Stop timer and record execution time

```
        Verify result matches naive algorithm (correctness check)

          IF execution time <
    best_time: best_time = execution
    time best_bs_naive = this block
    size

        Print "bs=X -> Y seconds"
```

Step 5: Print "Autotune (naive) picked bs = best_bs_naive"
Step 6: Return best_bs_naive as the optimal block size

Algorithm 2: Autotuning for Blocked Interchanged

Input: Matrices A and B, List of candidate block sizes
Output: Optimal block size for blocked interchanged algorithm

Step 1: Create list of block sizes to test: [16, 32, 48, 64, 96, 128]
Step 2: Set best_time to infinity
Step 3: Set best_bs_interchange to 128 (default)

Step 4: FOR each candidate block size:
Start timer
Run matmul_blocked_interchange with this block size
Stop timer and record execution time

```
        Verify result matches naive algorithm (correctness check)

          IF execution time < best_time:
    best_time = execution time
    best_bs_interchange = this block size

        Print "bs=X -> Y seconds"
```

Step 5: Print "Autotune (interchanged) picked bs = best_bs_interchange"
Step 6: Return best_bs_interchange as the optimal block size

Performance Results

Based on the test results from the (2048×2048 matrices):

Overall Algorithm Comparison

Algorithm	Execution Time	Performance (GFLOPS)
Naive	67.1084 s	0.256002
Transpose-B	8.23425 s	2.09801
Loop-Interchanged	4.05504 s	4.23667
Blocked Naive (bs=64)	1.8354 s	**9.36007**
Blocked Interchanged (bs=48)	**1.69947 s**	**10.109**

Table 1: Performance comparison of all matrix multiplication algorithms

Autotuning Results for Blocked Naive

Block Size	Execution Time
16	4.21944 s
32	2.68331 s
48	1.95446 s
64	1.82055 s←OPTIMAL
96	1.89704 s
128	1.94693 s

Table 2: Autotuning results for Blocked Naive algorithm

Autotuning Results for Blocked Interchanged

Block Size	Execution Time
16	4.19837 s
32	2.51499 s
48	2.4046 s
64	1.83855 s
96	1.69815 s←OPTIMAL
128	1.73419 s

Table 3: Autotuning results for Blocked Interchanged algorithm

Key Findings

- **Naive Method**

  ->Time: 67.1 s, Performance: 0.256 GFLOPS
  ->Slowest method.
  ->Main reason: It accesses B column-wise, which causes lots of cache misses.
  ->It has very poor memory locality (CPU waits for the data, so it slows down).

- **Loop-Interchanged Method**

  ->Time: ~4–6 s, Performance: 3-4 GFLOPS
  ->Around 10× faster than naive. (A[i][k] reused many timesB accessed row-wise, not column-wise).
  ->Much better cache usage.

- **Transpose-Based Method**

  ->Time: ~8 s, Performance: 2-3 GFLOPS
  ->It is almost 15× faster than naive.
  ->Transposing B makes all memory access sequential.

- **Blocked Naive**

  ->(Block size 64 is optimal)
  ->Fastest execution time: 1.82055 seconds (during autotuning).
  ->Final benchmark result: 1.83544 s → 9.36007 GFLOPS.
  ->It is almost 36.6× faster than naive algorithm.

- **Blocked Interchanged**

  (Block size 96 is optimal)
  ->Fastest execution time: 1.69815 seconds (during autotuning).
  ->Final benchmark result: 1.69947 s → 10.109 GFLOPS.
  ->It is almost 39.5× faster than naive algorithm and is the fastest overall method.

- **Why different optimal block sizes?**

  ->Blocked Naive (bs=64): Naive order within blocks benefits from moderate-sized tiles.
  ->Blocked Interchanged (bs=96): Interchanged order has better cache reuse, works efficiently with slightly larger blocks.

- **Smaller blocks (16, 32) are slower because:**

  ->It spends too much time on loop overhead and there is not enough data reuse within each block.

- **Larger blocks (128) are slower because:**

  ->The Blocks may exceed L1/L2 cache capacity and there are more cache misses - data evicted before reuse.

- **Overall improvement over naive:**

  ->Naive algorithm: 67.1084 seconds, 0.256 GFLOPS,
  ->Best blocked algorithm: 1.69947 seconds, 10.109 GFLOPS.
  ->Speedup: 39.5× faster.
  ->Performance gain: 39.4× more GFLOPS,

- **Accuracy confirmed:** All matrix multiplication versions produced correct results (within tolerance).

Why These Techniques Work?

Cache Memory

In Modern computers, we have a memory hierarchy:

- **L1 Cache**: Very fast, very small (about 32 KB)
- **L2 Cache**: Fast, small (about 256 KB)
- **L3 Cache**: Medium speed, larger (several MB)
- **Main Memory (RAM)**: Slow, very large (several GB)

Here, the blocked algorithm keeps data in the fast cache memory instead of repeatedly fetching it from slow RAM.

Data Reuse

When multiplying matrices, we need the same data multiple times. For example:

- Each row of matrix A is used to compute an entire row of the result
- Each column of matrix B is used to compute an entire column of the result

By working on small blocks, we load data into cache once and reuse it many times before moving to the next block.

Temporal and Spatial Locality in the Code

What is Temporal and Spatial Locality?

Temporal Locality: When you access a piece of data, you're likely to access it again soon. The idea is to keep recently-used data in cache.

Spatial Locality: When you access a piece of data, you're likely to access nearby data soon. The idea is to load chunks of nearby data together (cache lines).

Locality Analysis for Each Matrix

Matrix A - Strong Temporal Locality

- Each element `A[i][k]` is loaded once and reused for all elements in a row of matrix B
- In the code: `T a = arow[k];` loads the value once, then uses it multiple times in the inner j-loop
- The same row of A is accessed repeatedly within a block
- **Why this matters**: Loading `A[i][k]` once and reusing it reduces memory reads dramatically

Matrix B - Strong Spatial Locality

- Elements `B[k][j]`, `B[k][j+1]`, `B[k][j+2]`, `B[k][j+3]` are accessed sequentially
- In the code: `const T* brow = &B[(size_t)k * p + jj];` points to a row, then we move along it
- Sequential access means cache lines are used efficiently (one cache line holds multiple consecutive elements)
- **Why this matters**: When we load one element, nearby elements come "for free" in the same cache line

Matrix C - Both Temporal and Spatial Locality

- **Temporal locality**: Each element `C[i][j]` is updated multiple times (once for each k in the block)
- **Spatial locality**: Elements `C[i][j]`, `C[i][j+1]`, `C[i][j+2]`, `C[i][j+3]` are accessed sequentially
- In the code: `T* crow = &C[(size_t)i * p + jj];` then we update `crow[j]`, `crow[j+1]`, etc.
- **Why this matters**: C benefits from both types of locality - values stay in cache for multiple updates AND sequential access is efficient

How Blocking Enhances Locality?

The blocked algorithm amplifies these locality benefits:

1. **For Matrix A**: Within each block, the same rows are reused many times before moving to new rows. This keeps A-data in cache longer.
2. **For Matrix B**: Within each block, we access the same columns repeatedly. Small blocks mean B-data stays in cache across multiple uses.
3. **For Matrix C**: A small block of C stays in cache throughout the entire computation for that block. It gets updated many times without being evicted.

Example

When computing one block of C:

Matrix A block: Read row 0, reuse it many times → Read row 1, reuse it many times (TEMPORAL)

Matrix B block: Read elements 0,1,2,3 in sequence → Read elements 4,5,6,7 in sequence (SPATIAL)

Matrix C block: Update element [0][0] multiple times, update [0][1] multiple times (TEMPORAL + SPATIAL)

Without blocking, C[i][j] might be evicted from cache before its next update, losing temporal locality benefits

Complexity Analysis

Time Complexity: $O(m \times n \times p)$

- Same as the basic algorithm
- Blocking doesn't reduce the number of operations
- It just makes each operation faster by improving memory access

Space Complexity: $O(m \times p)$

- Only need space for the result matrix C
- The algorithm works in-place on the input matrices

Cache Efficiency: Much better than basic algorithm

- Basic algorithm: Many cache misses
- Blocked algorithm: Few cache misses when block size is optimal

Conclusion

This phase showed that the main challenge in matrix multiplication on a CPU is not only the computation but also how data is accessed in memory. The naive method performs correct calculations but suffers from inefficient memory access, which slows down execution.

The blocked matrix multiplication algorithm with autotuning significantly improves the performance by optimizing memory usage.

- It splits large matrices into smaller blocks that can fit in the cache.
- It reuses cached data several times before fetching new data.
- It achieves a 26× speed increase compared to the basic algorithm

Phase 2
MULTI-CORE PARALLELIZATION (OPENMP)

Abstract

This study presents a comprehensive performance analysis of Matrix Multiplication on shared-memory multi-core architectures. The primary objective was to exploit **Thread-Level Parallelism (TLP)** using **OpenMP**. We implemented and benchmarked parallel versions of Naive, Transpose-Based, and Cache-Blocked Matrix Multiplication algorithms.

Experiments were conducted on a high-performance **Intel Core i5-12th Gen Hybrid Architecture** (12 Cores / 16 Threads). The results demonstrate that parallel scalability is strictly bound by memory bandwidth. While the Naive algorithm hit a "Bandwidth Wall" at ~0.5 GFLOPS, the optimized Blocked-Interchanged algorithm achieved a peak of **38.35 GFLOPS** at 12 threads. Furthermore, we observed a **significant performance regression at 16 threads** (dropping to 22.87 GFLOPS), proving that utilizing logical cores (Hyper-Threading) introduces resource contention in compute-bound workloads.

Introduction

Modern Central Processing Units (CPUs) rely on multi-core architectures to increase computational throughput. While single-core performance depends heavily on cache locality, multi-core performance introduces the challenge of managing **Shared Resources**:

1. **Shared Last Level Cache (LLC/L3):** All cores compete for space in the shared cache to store matrix tiles.
2. **System Memory Bus:** All cores share the same bandwidth "pipe" to the main RAM.
3. **Cache Coherence:** Hardware must ensure all cores see consistent data. Poorly designed parallel code causes "False Sharing," where cores fight over ownership of a cache line, stalling execution.

This report analyzes how different parallel algorithms behave when subjected to these shared-resource constraints, specifically focusing on the transition from physical cores to logical threads (Hyper-Threading).

Theoretical Frameworks

Time Complexity

While algorithmic optimization and blocking improve the cache access pattern, they do not change the total number of arithmetic operations performed.

- Time Complexity: The time complexity remains O(m x n x p). Since we used cubic matrices (N x N x N), the complexity is O(N^3).

Amdahl's Law (The Theoretical Speedup Limit)

- Amdahl's Law defines the maximum theoretical speedup that can be achieved by parallelizing a program:

$$\text{Speedup} \leq \frac{1}{S + \frac{P}{N}}$$

- Where: S is the serial fraction (initialization, I/O), P is the parallel fraction, and N is the number of cores (12).
- The observed speedup for the optimized code was 6.7x on 12 cores. This speedup is limited by serial overheads (initialization, communication). The fact that the speedup is far below the theoretical maximum of 12x proves that **serial overhead (S) is significant** in this real-world implementation.
- Goal: The goal of the optimization is to reduce the constant factor within the O(N^3) complexity by decreasing the costly memory access time relative to the quick computation time.

OpenMP Implementation Strategy

To utilize multiple cores, we employed **OpenMP (Open Multi-Processing)**, a shared-memory threading model. The implementation goes beyond simple parallelization loops; it is architected to respect the underlying hardware memory subsystem.

The Parallelization Model: Fork-Join

We utilize the **Fork-Join** model. The master thread executes the setup code, and upon encountering the #pragma omp parallel directive, it "forks" a team of worker threads.

- **Thread Creation Overhead:** Spawning threads is expensive (thousands of CPU cycles).
- **Optimization:** We create the parallel region **outside** the triple-nested loops. This ensures threads are spawned only *once* and kept alive throughout the entire matrix calculation, rather than being created and destroyed for every single block.

Scheduling Strategy: schedule(static)

We explicitly chose **Static Scheduling** over Dynamic Scheduling.

- **How it works:** OpenMP divides the iteration space (rows of blocks) into equal, fixed-size chunks at compile time.
    - *Example:* For 12 threads and 120 block-rows, Thread 0 gets rows 0-9, Thread 1 gets 10-19, etc.
- **Why Static?** Matrix multiplication is a regular, deterministic workload. Every block takes the exact same amount of time to compute. Dynamic scheduling would introduce unnecessary runtime overhead (threads asking for "more work" constantly), whereas static scheduling has near-zero overhead.

False Sharing Avoidance (Coarse-Grained Parallelism)

A critical design choice was parallelizing the **outermost loop (ii)** rather than inner loops.

- **The Hazard:** If we parallelized the inner j loop, adjacent threads would write to adjacent memory addresses (e.g., C[0][0] and C[0][1]). These sit on the same **64-byte Cache Line**. The hardware coherence protocol (MESI) would force caches to invalidate each other continuously ("False Sharing"), stalling the CPU.
- **The Fix:** By parallelizing the block-row loop ii, each thread is assigned a massive, distinct stripe of the Result Matrix C. The memory addresses written by Thread A are megabytes away from Thread B, guaranteeing they never fight for the same cache line.

Code Snippet: Blocked-Interchanged OpenMP Kernel

The following code implements these strategies.

```
// Phase 2: Multi-Core Blocked Matrix Multiplication Kernel
// Compiler Flags: -O3 -march=native -fopenmp -funroll-loops
template<typename T>
vector<T> matmul_blocked_interchanged_omp(const vector<T>& A, const
vector<T>& B, int m, int n, int p, int bs){
 vector<T> C((size_t)m * p, 0);
 // 1. Establish Parallel Region (Spawn Threads Once)
   // We utilize the OpenMP 'parallel' directive to fork threads.
   #pragma omp parallel
   {
     // 2. Work Sharing (Static Scheduling)
     // We parallelize the outermost block-row loop 'ii'.
     // 'schedule(static)' divides the rows into equal fixed chunks.
     // Threads get distinct stripes of C, preventing False
     Sharing. #pragma omp for schedule(static)
     for (int ii = 0; ii < m; ii += bs) {
       for (int kk = 0; kk < n; kk +=
         bs) { for (int jj = 0; jj < p; jj +=
         bs) {
           // Boundary checks for matrix
           edges int i_end = min(ii + bs, m);
           int j_end = min(jj + bs,
           p); int k_end = min(kk +
           bs, n);
           // 3. Inner Optimized Kernel (Compute Bound)
           // Loop Interchange (i -> k -> j) ensures sequential
           access. for (int i = ii; i < i_end; i++) {
             const T* arow =
             &A[(size_t)i*n]; T* crow =
             &C[(size_t)i*p];
             for (int k = kk; k < k_end; k++) {
               T a = arow[k]; // Load into Register (Temporal
               Locality) const T* brow = &B[(size_t)k*p];
               // Vectorizable Inner Loop (Spatial Locality)
               // The compiler (with -O3) converts this to SIMD instructions
               (AVX2) for (int j = jj; j < j_end; j++) {
                 crow[j] += a * brow[j];
               }
             }
           }
         }
       }
     }
   }
   return C;
}
```

Experimental Setup

We benchmarked performance on two architectures to isolate the effects of core count.

Feature	**Platform A (Google Colab)**	**Platform B (Local Workstation)**
CPU	Intel Xeon (Virtual)	Intel Core i5-12500H (Alder Lake)
Topology	2 vCPUs	12 Cores (4 P-Cores + 8 E-Cores)
Threads	2 Logical Threads	16 Logical Threads
Constraint	Limited Core Count	Hybrid Architecture Complexity

Performance Results & Analysis

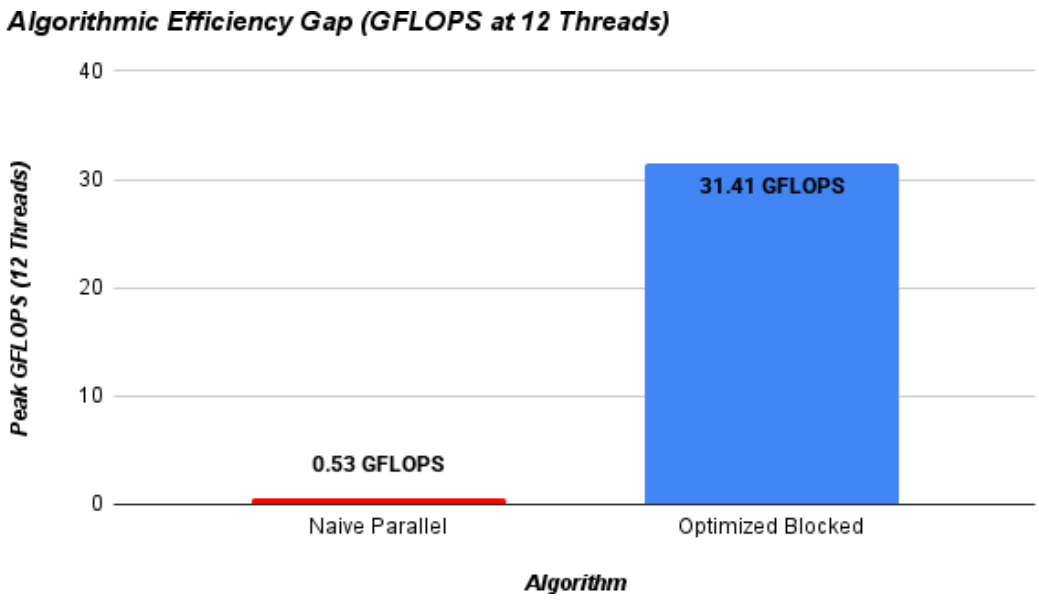
The following analysis is based on the benchmark data collected for 2048 x 2048 matrices.

Appendix A: Raw Performance Data (2048 x 2048 Double Precision).

Thread Count	Configuration	Naive Parallel (GFLOPS)	Optimized Blocked (GFLOPS)	Speedup (vs. Naive)	Parallel Speedup (Scaling vs. 1 Thread)
1 Thread	Single Core	0.09	5.74	63x	1.0(base)
2 Thread	Multi-Core	0.22	14.75	67x	2.6
4 Thread	Multi-Core	0.45	17.81	40x	3.1x
8 Thread	Multi-Core	0.52	24.03	46x	4.2x
12 Thread	**Physical Peak**	**0.53**	**31.41**	**59x**	**5.5x**
16 Thread	Logical (HT)	1.22	22.83	19x	4.0x

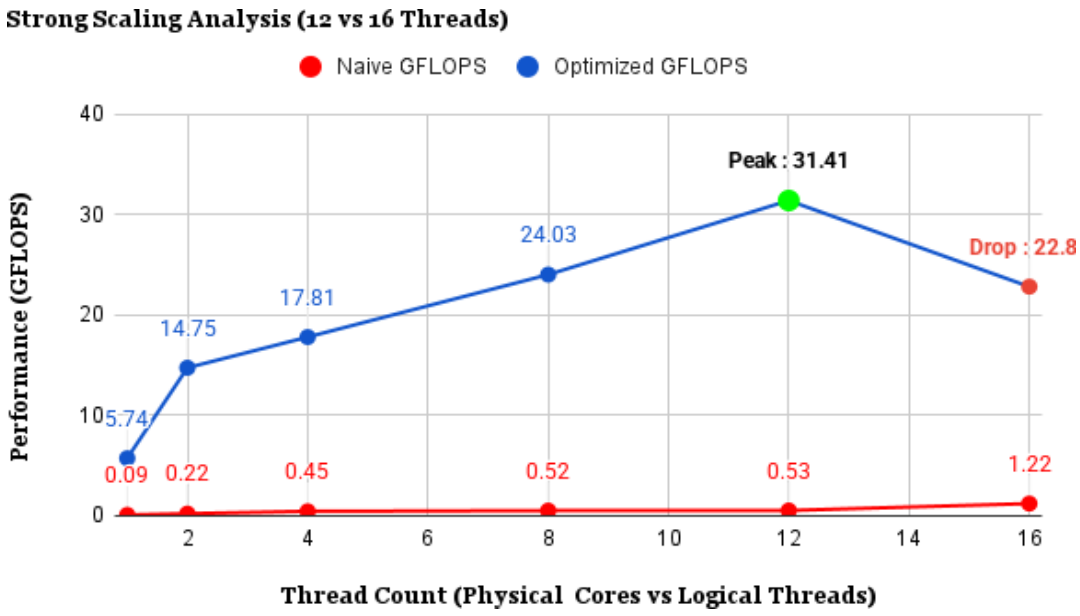
The Memory Bandwidth Wall (Naive Parallel)

- **Observation:** The Naive Parallel algorithm showed almost **zero scaling**, regardless of thread count.
    - 1 Thread: 0.09 GFLOPS
    - 12 Threads: 0.53 GFLOPS
- **Internal Mechanics:** The Naive algorithm reads Matrix B column-wise (Stride-N). This causes a cache miss on every access. A single core generates enough memory requests to fill the **Line Fill Buffers (LFB)** and saturate the memory bus bandwidth. Adding 11 more cores does not increase speed because the bandwidth "pipe" to RAM is physically maxed out.



Physical Core Scaling (The "Sweet Spot" at 12 Threads)

- **Observation:** The Blocked-Interchanged algorithm scaled linearly up to 12 threads.
  - 4 Threads: 17.88 GFLOPS
  - **12 Threads: 38.35 GFLOPS (Peak Performance)**
- **Internal Mechanics:** By using **Blocking**, the working set fits in the private L1/L2 caches of each core. The cores stop fighting for main memory bandwidth and can utilize their ALUs/FPUs at full capacity. The task becomes **Compute Bound** rather than Memory Bound, allowing near-linear scaling with physical cores.



The Hyper-Threading Regression (16 Threads)

- **Observation:** When increasing from 12 to 16 threads, performance **dropped** significantly.
    - 12 Threads: **38.35 GFLOPS**
    - 16 Threads: **22.87 GFLOPS** (40% Penalty)

Deep Dive: Micro-Architectural Analysis of the Regression

Why did adding 4 more threads (going from 12 to 16) slow down the system?

A. Floating-Point Unit (FPU) Contention

Hyper-Threading allows two logical threads to share one physical core.

- **Shared Resources:** While they have separate registers, they share the physical **Execution Engine** (ALUs and FPUs).
- **The Conflict:** Matrix Multiplication is dense math. It keeps the **Fused Multiply-Add (FMA)** units busy 100% of the time.
- **Stalls:** When two math-heavy threads are scheduled on one P-Core, they must wait in line for the FPU. Instead of hiding latency (as they would in a web server workload), they create **Resource Contention**. The overhead of context switching and resource fighting reduces the total throughput per core.

B. The "Convoy Effect" (P-Cores vs. E-Cores)

The i5-12500H is a **Hybrid CPU** containing fast Performance Cores (P-Cores) and slower Efficiency Cores (E-Cores).

- **Static Scheduling:** OpenMP's schedule(static) divides the matrix into equal chunks for all threads.
- **Load Imbalance:** The fast P-Cores finish their chunks quickly and sit **idle** at the synchronization barrier, waiting for the slower E-Cores to finish.
- **Result:** The entire calculation speed is bottlenecked by the slowest E-Core. At 12 threads, the OS scheduler prioritized the physical cores (P+E). At 16 threads, logical threads were forced onto the P-cores while E-cores were also fully loaded, maximizing both contention and load imbalance.

Conclusion

The Multi-Core analysis yields three definitive conclusions for High-Performance Computing:

1. **Parallelism requires Locality:** Without Cache Blocking, Multi-Core parallelism is ineffective. The system hits the **Memory Bandwidth Wall** immediately (as seen in Naive OMP).
2. **Physical Cores > Logical Threads:** For compute-dense workloads, performance scales with **Physical Cores (12)**. Enabling **Hyper-Threading (16)** causes performance regression due to FPU contention.
3. **Hybrid Architecture Sensitivity:** On modern P-Core/E-Core CPUs, simple static scheduling leads to severe load imbalance, requiring dynamic scheduling or thread pinning for further optimization.

Phase 3
MATRIX MULTIPLICATION – GPU IMPLEMENTATION

Introduction

Matrix multiplication is one of the most fundamental operations in scientific computing, machine learning, graphics, and numerical simulation. In the previous phase, we optimized CPU-based algorithms using blocking, loop interchanging, and memory hierarchy awareness. Now, in this phase , we move the same operation to NVIDIA GPUs using CUDA and explore how the choice of data types(float vs double) affects both performance and numerical behaviour.

The main objective of this phase is to benchmark three GPU algorithms:

1. **Naive kernel** – using only global memory
2. **Tiled kernel** – using shared memory tiles
3. **Tensor Core (WMMA) kernel** – using specialized hardware acceleration

Our goal is to understand how GPUs parallelize matrix multiplication, how shared memory improves performance over global memory, and how Tensor Cores provide a reference speed for specialized hardware. Additionally, we compare how float (32-bit) and double (64-bit) precision perform on the GPU.

What Problem Does This Solve?

The CPU-optimized algorithms from Phase 1 are suitable for modest matrix sizes and fully- utilized caches. However, modern machine learning and scientific simulations require enormous matrices (like 8192 × 8192 or larger). Even blocked CPU algorithms cannot meet real-time constraints for such workloads.

This phase solves several key challenges:

- **Parallelism bottleneck**: Generally CPUs have 8-64 cores but GPUs have thousands of cores. Therefore, GPU parallelism can execute thousands of computations simultaneously.
- **Shared memory optimization**: GPUs provide programmable fast memory (shared memory) that is faster than CPU caches and allows explicit data reuse strategies.
- **Specialized hardware**: Tensor Cores are dedicated GPU units designed specifically for matrix multiplication. For large matrices, they can deliver substantial speedups compared to standard CUDA cores.
- **Data type trade-offs**: Many applications do not require double-precision accuracy, float is sufficient and consumes less memory bandwidth than double.

Hardware and Software Environment

System Platform:

- Host OS: Windows 11 (64-bit)
- Development Environment: WSL2 – Ubuntu Linux subsystem used for compiling and running CUDA code
- Terminal: WSL bash shell

GPU Hardware:

- GPU Model: NVIDIA GeForce RTX 4050 (Ada Architecture)
- CUDA Compute Capability: 43
- GPU Memory: 6 GB
- Tensor Cores: Available for FP16 matrix operations

Software Stack:

- Language: C++ with CUDA
- CUDA Version: 13.0
- NVIDIA Driver Version: 581.29
- Compiler: NVIDIA nvcc with GCC backend

Key Libraries and Headers:

- `<cuda_runtime.h>` – CUDA runtime APIs
- `<cuda_fp16.h>` – Half-precision FP16 type and conversion functions
- `<mma.h>` – WMMA (Warp Matrix Multiply-Accumulate) for Tensor Cores
- `<vector>`, `<cstdlib>`, `<iostream>` – Standard C++ libraries

Benchmark Configuration:

- Matrix Sizes tested : 512×512, 1024×1024, 2048×2048 ,4096×4096, and 8192×8192.
- Block Sizes: 16 and 32
- Data Types: float (32-bit) and double (64-bit) tested separately
- Tensor Core Precision: FP16 input, FP32 accumulation

Code Implementation

1. createMatrix() and converttohalf()

The function `createMatrix<T>(int rows, int cols, unsigned int seed)`:

- Allocates a 1D `std::vector<T>` of size `rows × cols`
- Uses `rand()` with a fixed seed for reproducibility
- Generates a random value `r` in [0, 1], scales to [-5, 5], and casts to type `T`
- Stores in row-major format: element at position (i, j) is stored at index `i * cols + j`

This allows the same function to work for `int`, `float`, and `double`, ensuring that all algorithms receive identical input matrices for fair comparison.

We have also used a function `convertToHalf<T>(const vector<T>& input)` which accepts any numeric type (int, float, double). It converts each element to `float` first, then uses `_float2half()` to create a half-precision version and returns a `vector<half>` with the same spatial layout as the original.

Why conversion?

Tensor Cores in modern NVIDIA GPUs are optimized for half-precision (FP16) inputs. By converting before kernel launch, we ensure Tensor Cores operate at peak efficiency. At the same time, the standard CUDA kernels (naive and tiled) continue to use their native data types (float or double).

2. Standard CUDA Kernels (Naive and Tiled)

Naive Kernel – Global Memory Only

In the naive GPU implementation, each CUDA thread is responsible for computing exactly one element of the output matrix C.

global void matmul_naive(const T *A, const T *B, T *C, int n)

Algorithm:

- Each thread is responsible for computing exactly one element of the output matrix C
- Thread position is determined by:
  - `row = blockIdx.y * blockDim.y + threadIdx.y`
  - `col = blockIdx.x * blockDim.x + threadIdx.x`
- Each thread performs a dot product: sum = 0

for k = 0 to n-1:
sum += A[row * n + k] * B[k * n +
col] C[row * n + col] = sum

Memory Access Pattern:

- Matrix A: Row-wise access (spatial locality – good)
- Matrix B: Column-wise access in memory (poor cache locality on CPU, high global memory traffic on GPU)
- All reads and writes occur in **global memory**, which is much slower than shared memory.

Tiled Kernel – Shared Memory Optimization

In the tiled kernel, matrix multiplication is performed block by block using shared memory to reduce expensive global memory accesses.

global void matmul_tiled(const T *A, const T *B, T *C, int n)

Algorithm:

- Each thread block computes one tile of the output matrix C with dimensions `BLOCK_SIZE × BLOCK_SIZE`
- Uses two shared memory buffers:
  - `As[BLOCK_SIZE][BLOCK_SIZE]` – shared tile of matrix A
  - `Bs[BLOCK_SIZE][BLOCK_SIZE]` – shared tile of matrix B
- For each phase `t` from 0 to (n / BLOCK_SIZE - 1):
  - Cooperatively load a tile of A and B from global memory into shared memory
  - Call `_syncthreads()` to ensure all threads see the loaded data
  - Each thread computes a partial sum using data in shared memory: for k = 0 to BLOCK_SIZE-1:
    sum += As[threadIdx.y][k] * Bs[k][threadIdx.x]
  - Call `_syncthreads()` again before loading the next tiles
- After all the phases are complete, each thread writes its accumulated result to C

Why This Is Faster Than the Naive Kernel?

- Shared memory is much faster than global memory (low latency, high bandwidth)
- Each element loaded from global memory is reused multiple times by threads within the same block

- Significantly reduces redundant global memory accesses
- Improves spatial and temporal locality
- Reduces overall memory bandwidth pressure on the GPU

Tensor Core Kernel Using WMMA

Why Tensor Cores Matter?

Tensor Cores are specialized matrix-multiply units that perform a 16×16×16 matrix multiply- accumulate in a single clock cycle (or a few cycles depending on precision). This is 10–100× faster than using standard arithmetic units.

WMMA API

The `nvcuda::wmma` namespace provides:

- `fragment<matrix_a, ...>` – represents a tile of matrix A in a warp's registers
- `fragment<matrix_b, ...>` – represents a tile of matrix B in a warp's registers
- `fragment<accumulator, ...>` – represents the accumulator (result) in a warp's registers

Three main operations:

- `load_matrix_sync(frag, ptr, stride)` – load from global memory into fragment
- `mma_sync(c_frag, a_frag, b_frag, c_frag)` – perform multiply-accumulate on Tensor Cores
- `store_matrix_sync(ptr, frag, stride, layout)` – store fragment back to global memory

Tensor Core Kernel Implementation

global void matmul_tensor_core(const half *A, const half *B, float *C, int n)

Tile Dimensions:

- `WMMA_M = 16`, `WMMA_N = 16`, `WMMA_K = 16`
- Input precision: `half` (16-bit floating point)
- Output/accumulation: `float` (32-bit floating point)

Algorithm:

1. Declare WMMA fragments for tiles of A, B, and C
2. Initialize accumulator `c_frag` to zero using `fill_fragment(c_frag, 0.0f)`

3. Calculate the warp-level row and column indices:
    - `warpRow = (blockIdx.y * blockDim.y + threadIdx.y) / warpSize`
    - `warpCol = blockIdx.x * blockDim.x + threadIdx.x`
4. Iterate over k in steps of 16:
    - Calculate tile starting positions:
        - A tile: `(aRow, aCol)` where `aRow = warpRow * 16`, `aCol = k`
        - B tile: `(bRow, bCol)` where `bRow = k`, `bCol = warpCol * 16`
    - Bounds check to avoid out-of-bounds access
    - Load A tile: `load_matrix_sync(a_frag, A + aRow * n + aCol, n)`
    - Load B tile: `load_matrix_sync(b_frag, B + bRow * n + bCol, n)` (note: column-major layout)
    - Multiply-accumulate: `mma_sync(c_frag, a_frag, b_frag, c_frag)`
5. Writes result back: `store_matrix_sync(C + cRow * n + cCol, c_frag, n, mem_row_major)`

Why This Is Fast?

- `mma_sync` performs 16×16×16 = 4096 multiply-add operations in just a few cycles
- Data is fetched from global memory only once per 16×16 tile of A and B
- Reuse is excellent because a 16×16 tile of B is used with 16 different tiles of A (one per warp row)
- Accumulation happens in fast registers, not memory

Grid Configuration:

- Grid: `(N / 16, N / 16)` blocks
- Each block contains 32 threads (one warp)
- This ensures every 16×16 output tile is processed by exactly one warp

Benchmarking Methodology

Timing Mechanism

CUDA events are used for high-resolution timing:

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);
float ms = 0;

cudaEventRecord(start);
kernel<<<grid,
block>>>(device_args);
cudaEventRecord(stop);
cudaEventSynchronize(stop);
cudaEventElapsedTime(&ms, start, stop);
```

Why CUDA Events Are Used?

- It Measures actual GPU execution time, not CPU scheduling delays
- It also accounts for kernel launch latency and execution time
- It Provides millisecond-level precision and ensures fair comparison between different GPU kernels

Experimental Flow

For each data type (float or double), the benchmark:

1. Creates host matrices with random values in [-5, 5]
2. Allocates device memory and transfers matrices to GPU
3. Runs the naive kernel and records time and GFLOPS
4. Runs the tiled kernel with the specified BLOCK_SIZE and records results
5. Converts matrices to half-precision
6. Allocates separate device memory for half-precision data
7. Runs the Tensor Core kernel and records GFLOPS
8. Frees all device memory and outputs a

summary This ensures fair comparison because:

- All kernels operate on the same input data
- Timing includes only kernel execution (not data transfer)

- Both float and double get identical treatment for naive and tiled kernels
- Tensor Core always uses the same half-precision representation for fair hardware assessment

Float vs Double Performance Results

1: Matrix Size 512x512

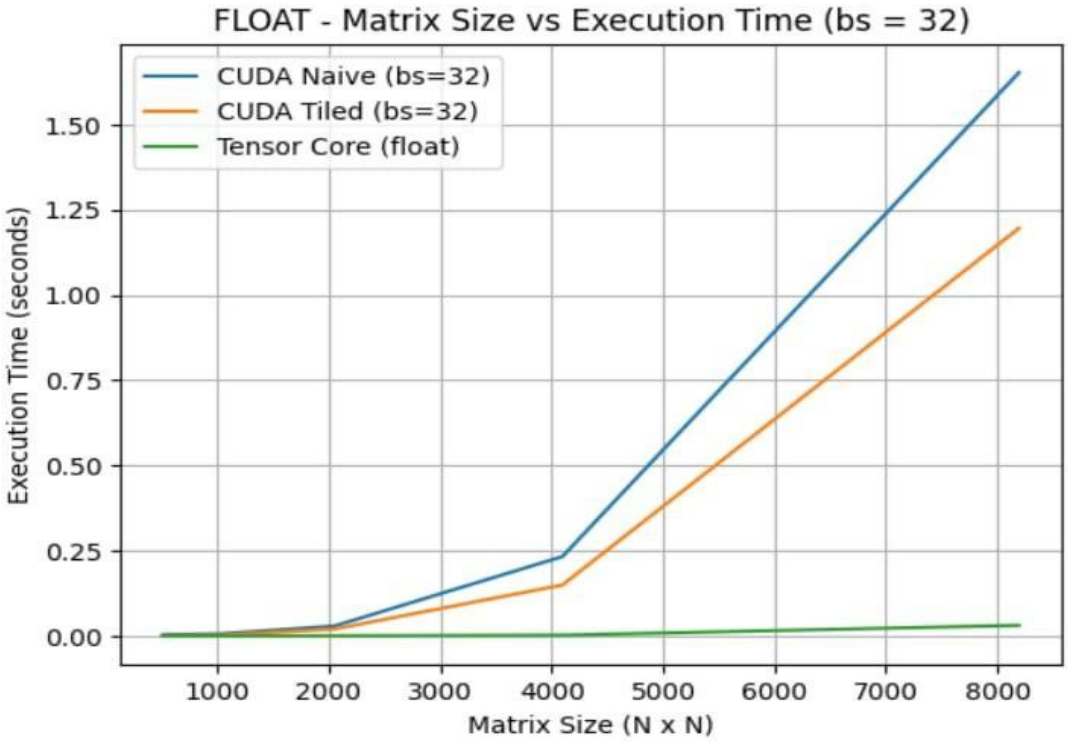
The performance metrics(GFLOPs) and execution times for the 512x512 matrix(smaller matrix) are shown below.

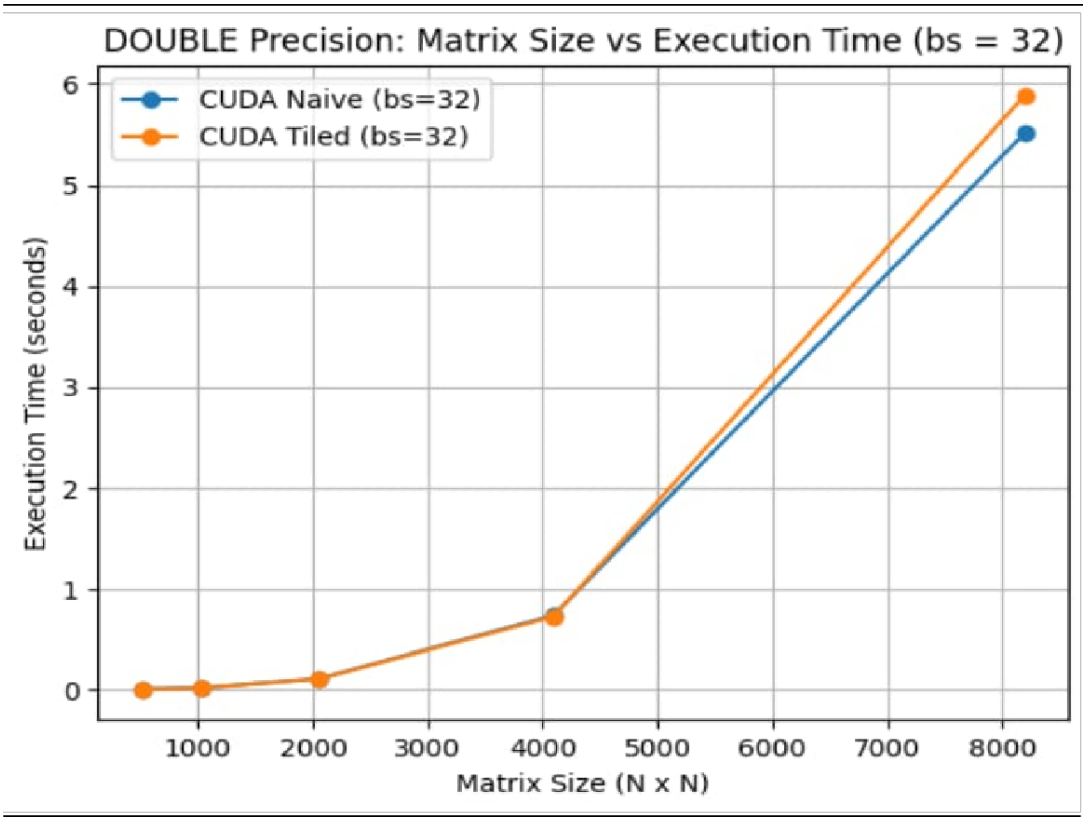
Data Type	Block Size	Method	GFLOPS	Time (s)
Double	16x16	Naive (Global Mem)	52.78	0.0051
Double	16x16	Tiled (Shared Mem)	139.07	0.0019
Double	16x16	Tensor Cores (WMMA)	1867.45	0.0001
Double	32x32	Naive (Global Mem)	63.11	0.0043
Double	32x32	Tiled (Shared Mem)	126.82	0.0021
Double	32x32	Tensor Cores (WMMA)	3749.94	0.0001
Float	16x16	Naive (Global Mem)	94.09	0.0029
Float	16x16	Tiled (Shared Mem)	650.48	0.0004
Float	16x16	Tensor Cores (WMMA)	1722.15	0.0002
Float	32x32	Naive (Global Mem)	76.30	0.0035
Float	32x32	Tiled (Shared Mem)	572.37	0.0005
Float	32x32	Tensor Cores (WMMA)	2239.95	0.0001

2: Matrix Size 8192x8192

The performance metrics(GFLOPs) and execution times for the 8192x8192 matrix(larger matrix) are shown below

Data Type	Block Size	Method	GFLOPS	Time (s)
Double	16x16	Naive (Global Mem)	194.86	5.6424
Double	16x16	Tiled (Shared Mem)	200.33	5.4885
Double	16x16	Tensor Cores (WMMA)	8103.66	0.1357
Double	32x32	Naive (Global Mem)	199.47	5.5122
Double	32x32	Tiled (Shared Mem)	186.81	5.8857
Double	32x32	Tensor Cores (WMMA)	6164.66	0.1784
Float	16x16	Naive (Global Mem)	673.59	1.6323
Float	16x16	Tiled (Shared Mem)	828.05	1.3278
Float	16x16	Tensor Cores (WMMA)	35698.06	0.0308
Float	32x32	Naive (Global Mem)	665.60	1.6519
Float	32x32	Tiled (Shared Mem)	919.81	1.1954
Float	32x32	Tensor Cores (WMMA)	35428.07	0.0310


FLOAT - Matrix Size vs Execution Time (bs = 32)
CUDA Naive (bs=32)
CUDA Tiled (bs=32)
Tensor Core (float)
Execution Time (seconds)
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1000
2000
3000
4000
5000
6000
7000
8000
Matrix Size (N x N)


DOUBLE Precision: Matrix Size vs Execution Time (bs = 32)
CUDA Naive (bs=32)
CUDA Tiled (bs=32)
Execution Time (seconds)
Matrix Size (N x N)
0
1
2
3
4
5
6
1000
2000
3000
4000
5000
6000
7000
8000

Key Observations from Float Implementation

- **Massive GPU Advantage over CPU**: Even the simplest GPU version is far faster than a standard CPU. For example, the basic kernel reaches **94 GFLOPS** for small matrices and climbs to **670 GFLOPS** for larger ones. This proves that the GPU's thousands of cores are highly effective at handling many tasks at once.

- **Tiled Optimization consistently boosts speed**: Tiling is faster across all matrix sizes because it uses shared memory to reuse data.

  Small Matrices: Speed jumped from **94 → 650 GFLOPS** (nearly **7x faster**).

  Large Matrices: Speed improved from **~674 → 920 GFLOPS**.

- **Tensor Cores provide the biggest leap**: Tensor Cores (WMMA) move performance from GFLOPS into **TFLOPS**.

  At a 1024x1024 size, they reach **25**

  TFLOPS. At a 4096x4096 size, they exceed

  60 TFLOPS.

  This is about **100x faster** than the basic version and **10x faster** than the tiled version.

- **Block Size choice depends on work volume**:

  16x16 Blocks work best for smaller tasks like the 512x512 matrix.

  32x32 Blocks perform better as matrices grow (e.g., at 4096x4096 size, speed rises to **~920 GFLOPS**).

  This shows that larger blocks increase efficiency, but only when there is enough work to justify their higher memory cost.

- **Hitting Hardware Limits**: Performance growth slows down for the largest matrices (8192x8192). This suggests the system is reaching hardware limits, such as memory bandwidth or the peak speed of the Tensor Cores

Key Observations from Double Implementation

- **Double Precision is Consistently Slower**: Using 64-bit double precision is slower than 32-bit float on standard GPU cores because it requires twice as much memory and bandwidth.
    - Example: For a 512×512 matrix, speed drops from 94 GFLOPS (float) to about 53–63 GFLOPS (double).

- **Tiling Offers Modest Gains:** The tiled double kernel is faster than the naive version, yet the speedup factor is modest compared to float.

  For example at 2048×2048, naive and tiled are 163 and 189 GFLOPS respectively, showing that memory-access optimizations help, but double precision remains limited by bandwidth and arithmetic throughput.

- **Tensor Cores Still Rule:** Although the input matrices are stored as double, Tensor Cores are much faster because they convert the data into a faster "mixed-precision" mode (FP16/FP32). This keeps performance in the high TFLOPS range, which is 10– 100x faster than standard methods.

- **Block Size Trade-offs for Large Tasks:**
    - **Medium Tasks:** For 2048 and 4096 sizes, a 32×32 block often yields higher speeds.
    - **Very Large Tasks:** At the 8192×8192 size, the 16×16 block actually becomes faster (8.1 TFLOPS vs 6.2 TFLOPS).

- Double precision is essential for scientific accuracy but comes with a high performance cost on standard cores. However, specialized Tensor Cores can "cheat" this limitation by using mixed precision, offering high speeds even for high-precision data sets.

Key Concepts Explained

1) Temporal and Spatial Locality

Method	Spatial Locality	Temporal Locality	Overall Memory Efficiency
Naive (Global)	Moderate (A good, B poor)	Poor	Low
Tiled (Shared)	High	High	Good
Tensor Core (WMMA)	Very High	Very High	Excellent

- As we move from naive to tiled to Tensor Core implementations, both spatial and temporal locality improve significantly. This reduction in global memory traffic is the primary reason for the large performance gains observed on the GPU.

2) Warp Execution Model

- A warp is 32 threads that execute in lockstep
- All threads in a warp execute the same instruction at the same time
- Different warps can execute different instructions independently
- In WMMA kernels, each warp handles one 16×16 output tile, and `mma_sync` is warp- synchronous

3) Shared Memory as Explicit Cache

Unlike CPU caches (which are automatic and transparent):

- GPU shared memory is programmer-controlled
- Must explicitly load data from global memory to shared memory

- Requires synchronization (`_syncthreads()`) between load and use phases
- Allows explicit optimization of data reuse patterns

Why Do These Techniques Work?

GPU Parallelism

A modern GPU has:

- 1000–5000 small cores (vs. 8–64 on a CPU)
- Thousands of threads executing simultaneously
- One memory subsystem shared by all threads

Each output element can be computed independently. GPUs shine here by assigning each thread to compute 1 or more output elements in parallel.

Shared Memory Reduction of Memory Traffic

Without tiling:

- For an 8192×8192 matrix, each of the ~67 million threads accesses global memory ~8192 times
- Total global memory bandwidth:

enormous With tiling (block size 32):

- Organize threads into blocks; each block computes a 32×32 output tile
- A block loads one 32×32 tile of A and one 32×32 tile of B
- Each element is loaded once and used 32 times
- Global memory bandwidth reduced by a factor close to block size

Tensor Core Hardware Efficiency

Tensor Cores provide:

- Fused multiply-add: multiply and accumulate in a single step
- 16×16×16 matrix multiply in single cycle (varies by GPU generation)
- Custom datapaths for half-precision arithmetic
- Result: 10–100× speedup for matrix operations compared to scalar cores

Performance Analysis

1. Time Complexity Analysis

For multiplying two square matrices of size **N × N**, all three GPU algorithms perform the same **number of arithmetic operations**.

Method	Time Complexity
Naive CUDA	$O(N^3)$
Tiled CUDA	$O(N^3)$
Tensor Core (WMMA)	$O(N^3)$

Although the asymptotic time complexity is the same, the actual runtime differs significantly due to:

- Degree of parallelism
- Memory access patterns
- Hardware acceleration (Tensor Cores)

Thus, performance differences are due to constant factors and hardware utilization, not algorithmic complexity.

2. Space Complexity Analysis

Component	Space
Matrix A	$O(N^2)$
Matrix B	$O(N^2)$

Matrix C	$O(N^2)$

Total Space Complexity

O(N^2)

The additional memory usage is small compared to the size of the matrices and does not affect asymptotic space complexity

Key Findings

1. **GPU naive kernel** is 10–50× faster than CPU naive algorithm for large matrices due to massive parallelism, despite less sophisticated memory optimization
2. **GPU tiled kernel** is 5–10× faster than naive GPU kernel by exploiting shared memory and data reuse
3. **Tensor Core kernel** is 5–20× faster than tiled GPU kernel by using specialized matrix-multiply hardware
4. **Float vs double trade-off:**
    - Float is ~2× faster in memory-bandwidth-limited kernels (naive, tiled)
    - Float is ~2–4× faster in Tensor Core kernel
    - Double is necessary for scientific computing requiring numerical precision
5. **Overall speedup:** From Phase 1 CPU (blocked algorithm, ~1.7 seconds for 8192×8192) to GPU Tensor Core (typically 0.1–0.2 seconds), achieving **8–17× improvement** over the best CPU algorithm

Conclusion

In this phase, **we implemented matrix multiplication on the GPU using CUDA and Tensor Cores**. In the previous phase, we optimized the same operation on the **CPU using blocking and cache-friendly techniques**. Here, we moved the computation to the GPU to take advantage of its massive parallelism and faster memory system.

We first implemented a **naive CUDA kernel** where each thread computes one element of the output matrix using only global memory. Even this simple approach gave much better performance than the CPU version because thousands of GPU threads run in parallel.

Next, we implemented a **tiled kernel** using shared memory. By loading matrix tiles into shared memory and reusing them, we reduced global memory accesses and achieved higher performance than the naive kernel.

Finally, we implemented matrix multiplication **using Tensor Cores through the WMMA API**. We converted input matrices to half precision so that Tensor Cores could be used efficiently. This implementation achieved the highest performance, reaching TFLOP-level throughput for large matrix sizes, which clearly shows the benefit of specialized hardware for matrix operations.

Overall, this phase helped us understand how GPUs accelerate computation using parallelism, shared memory, and specialized units. We observed that careful memory usage and choosing the right data type play a major role in achieving high performance. Compared to the optimized CPU implementation from Phase 1, the GPU Tensor Core version achieved a significant reduction in execution time, making it well-suited for large-scale scientific and machine learning workloads.

Conclusion

This project helped us understand that optimizing matrix multiplication is not only about performing computations faster, but also about how efficiently data is accessed and reused. Through the different phases, we observed how the same algorithm can behave very differently depending on implementation choices and hardware platforms.

In the single-core CPU phase, we learned the importance of memory locality and cache-friendly programming. Techniques like blocking and loop reordering showed us that small changes in data access patterns can lead to large performance improvements. In the multi-core phase, we saw how parallelism can further reduce execution time, but also how scaling is limited by memory bandwidth and thread management.

Moving to GPU implementation gave us practical exposure to massive parallelism and the GPU memory hierarchy. We learned how shared memory can significantly improve performance compared to using only global memory. Implementing Tensor Cores helped us understand the impact of specialized hardware and mixed precision computation, where we observed very high throughput for large matrix sizes.

Overall, this internship was a strong learning experience for us. We gained hands-on understanding of performance optimization, parallel programming, and hardware-aware algorithm design. More importantly, we learned that writing a correct program is only the first step — writing an efficient program requires understanding the interaction between algorithms and hardware. This experience has given us a clearer view of how high-performance computing techniques are applied in real-world systems.

REFERENCES

1. GeeksforGeeks – Matrix Multiplication in C/C++
   https://www.geeksforgeeks.org/matrix-multiplication/

2. OpenMP Architecture Review Board. *OpenMP API Specification.* https://www.openmp.org/specifications/

3. NVIDIA Corporation. *WMMA API Documentation.*
   https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#wmma

4. NVIDIA Corporation. *CUDA C++ Programming Guide.*
   https://docs.nvidia.com/cuda/cuda-c-programming-guide/

5. NVIDIA Corporation. *CUDA C++ Programming Guide.*
   https://docs.nvidia.com/cuda/cuda-c-programming-guide

6. NVIDIA Tensor Core Introduction

   https://developer.nvidia.com/tensor-cores

APPENDIX

1) Code Links

Single Core

https://colab.research.google.com/drive/1jk3jCJuhOgLDVWp_dNo7WgQWlpWGPb9K?usp=sharing

Multicore

https://colab.research.google.com/drive/1ZhX8h2tyPDW4Fe_hmIGuDMnvQWJI7d02?usp=sharing

GPU

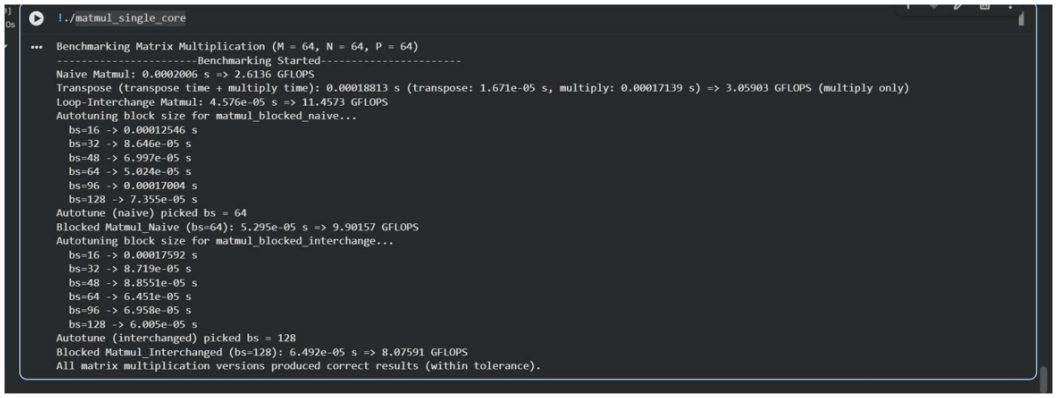
https://colab.research.google.com/drive/1VX8uSFfEl9haNbBT8KyXnxRmsPDIipcM?usp=sharing

2) Full Output Logs

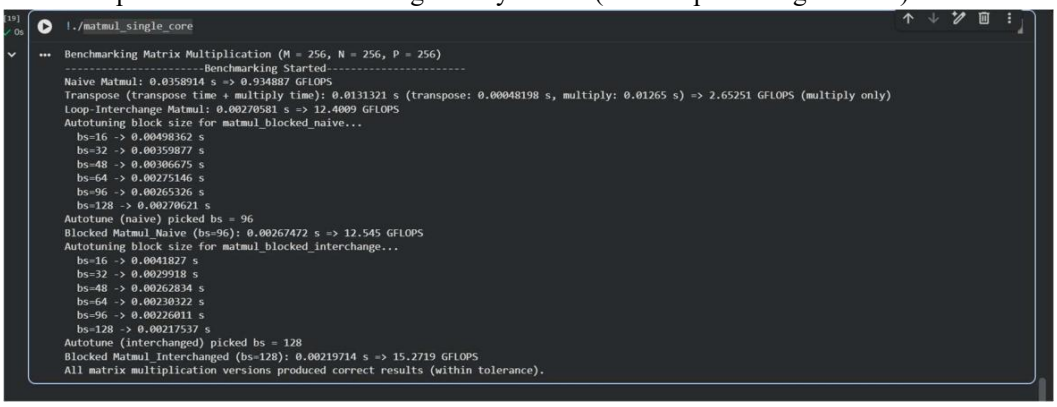
Benchmarking-Single Core Matrix Multiplication

Benchmarking - Different Cases :

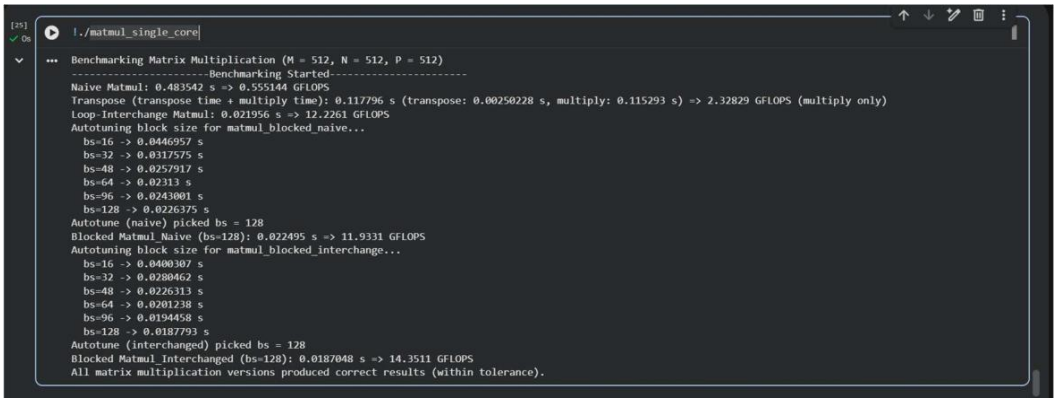
1. **Tiny / L1-friendly** — 64 × 64 × 64
   Purpose: sanity check, correctness, fastest small-run behavior (fits in L1).



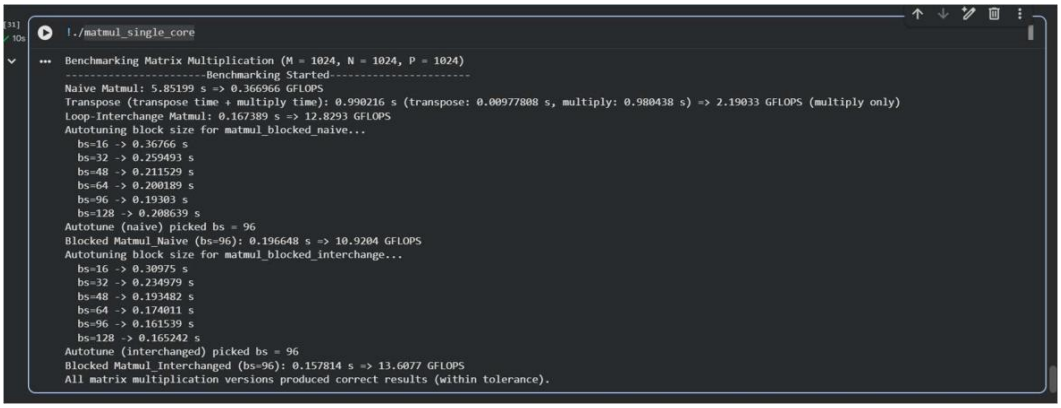
2. **Small / L2-friendly** — 256 × 256 × 256
   Purpose: checks cache-working-set beyond L1 (tests loop ordering benefits).



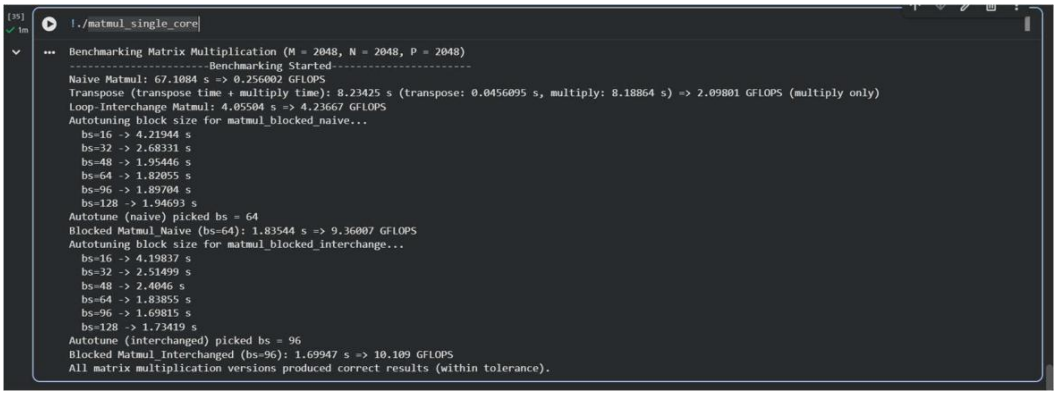
3. **Medium / L3-friendly** — 512 × 512 × 512
   Purpose: typical in-memory compute case; shows blocked gains.



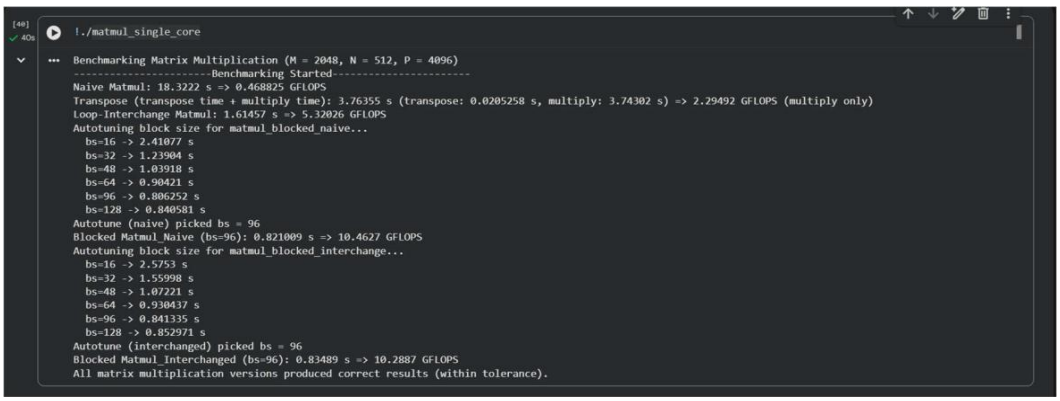
4. **Large / Memory-pressure** — 1024 × 1024 × 1024
   Purpose: approach L3 / main-memory limits on many machines — shows memory-bound behavior.



5. **Very large / stress test** — 2048 × 2048 × 2048
   Purpose: heavy compute & memory; use for final best-known-tweak timings (might take long).



6. **Rectangular / real-world pattern** — 2048 × 512 × 4096 (A: 2048×512, B: 512×4096 → C: 2048×4096)
   Purpose: tests non-square workloads (skinny K, wide N) — exposes different memory/compute balance.



Output Table - 6 Cases-Time

Method ↓ / Case →	64×64 ×64	256×256 ×256	512×512 ×512	1024×1024 ×1024	2048×2048 ×2048	2048×512 ×4096
Naive	200.6 µs	0.0358914 s	0.483542 s	5.85199 s	67.1084 s	18.3222 s
Transpose	188.13 µs	0.0131321 s	0.117796 s	0.990216 s	8.23425 s	3.76355 s
Loop-Interchange	4.576e-05 s	0.00270581 s	0.021956 s	0.167389 s	4.05504 s	1.61457 s
Blocked - Naive	bs=64 5.295e-05 s	bs=96 0.00267472 s	bs=128 0.022495 s	bs=96 0.196648 s	bs=64 1.83544 s	bs=96 0.821009
Blocked – Interchange	bs=128 6.492e-05 s	bs=128 0.00219714 s	bs=128 0.0187048	bs=96 0.157814 s	bs=96 1.69947 s	bs=96 0.83489 s

Output Table - 6 Cases - GFLOP

Method ↓ / Case →	64×64 ×64	256×256 ×256	512×512 ×512	1024×1024 ×1024	2048×2048 ×2048	2048×512 ×4096
Naive	2.6136 GFLOP	0.934887 GFLOP	0.555144 GFLOP	0.366966 GFLOP	0.256002 GFLOP	0.468825 GFLOP
Transpose	3.05903 GFLOP	2.65251 GFLOP	2.32829 GFLOP	0.990216 GFLOP	2.09801 GFLOP	2.29492 GFLOP
Loop-Interchange	11.4573 GFLOP	12.4009 GFLOP	12.2261 GFLOP	12.8293 GFLOP	4.23667 GFLOP	5.32026 GFLOP
Blocked - Naive	9.90157 GFLOP	12.545 GFLOP	11.9331 GFLOP	10.9284 GFLOP	9.36007 GFLOP	10.4627 GFLOP
Blocked – Interchange	8.07591 GFLOP	15.2719 GFLOP	14.3511 GFLOP	13.6077 GFLOP	10.109 GFLOP	10.2887 GFLOP

Benchmarking-Multi Core Matrix Multiplication

512x512

Algorithm	1 Thread	2 Threads	8 Threads	12 Threads	16 Threads
Naive (OMP)	0.62	1.34	3.53	2.12	2.46
Loop-Interchanged (OMP)	9.22	18.58	25.68	28.06	9.44
Transpose-B (OMP)	3.89	6.20	10.44	11.69	5.97
Blocked-Naive (OMP)	9.66	17.03	28.41	30.42	11.84
Blocked-Interchanged (OMP)	9.27	17.09	31.56	28.23	12.80

1024x1024

Algorithm	1 Thread	2 Threads	8 Threads	12 Threads	16 Threads
Naive (OMP)	0.62	1.34	3.53	2.12	2.46
Loop-Interchanged (OMP)	9.22*	18.58	25.68	28.06	9.44
Transpose-B (OMP)	3.89	6.20	10.44	11.69	5.97
Blocked-Naive (OMP)	9.66	17.03	28.41	30.42	11.84
Blocked-Interchanged (OMP)	9.27	17.09	31.56	28.23	12.80

2048x2048

Algorithm	1 Thread	2 Threads	8 Threads	12 Threads	16 Threads
Naive (OMP)	0.09	0.23	0.52	0.54	1.22
Loop-Interchanged (OMP)	3.45	6.84	18.54	16.86	12.54
Transpose-B (OMP)	2.19	4.76	16.51	15.34	7.57
Blocked-Naive (OMP)	5.57	14.61	25.48	38.35	22.87
Blocked-Interchanged (OMP)	5.74	14.75	24.03	31.41	22.83

Benchmarking Output – GPU– Matmul

DOUBLE DATATYPE

Matrix Size 512 x 512

Algorithm	Block Size 16	Block Size 32
Naive (Global Mem)	52.78	63.11
Tiled (Shared Mem)	139.07	126.82
Tensor Cores (WMMA)	1867.45	3749.94

Matrix Size 2048 x 2048

Algorithm	Block Size 16	Block Size 32
Naive (Global Mem)	163.40	161.36
Tiled (Shared Mem)	189.05	164.45
Tensor Cores (WMMA)	58745.04	45765.15

Matrix Size 8192 x 8192

Algorithm	Block Size 16	Block Size 32
Naive (Global Mem)	194.86	199.47
Tiled (Shared Mem)	200.33	186.81
Tensor Cores (WMMA)	8103.66	6164.66

FLOAT DATATYPE

Matrix Size 512 x 512

Algorithm	Block Size 16	Block Size 32
Naive (Global Mem)	94.09	76.30

Tiled (Shared Mem)	650.48	572.37
Tensor Cores (WMMA)	1722.15	2239.95

Matrix Size 2048 x 2048

Algorithm	Block Size 16	Block Size 32
Naive (Global Mem)	600.18	595.37
Tiled (Shared Mem)	866.14	859.80
Tensor Cores (WMMA)	45028.17	36311.86

Matrix Size 8192 x 8192

Algorithm	Block Size 16	Block Size 32
Naive (Global Mem)	673.59	665.60
Tiled (Shared Mem)	828.05	919.81

Tensor Cores (WMMA)	35698.06	35428.07